

Comparative Statistical Validation Report

Random Number Generator Analysis

BBRES-RNG | Java SecureRandom | Java Math.random()
30-Test Statistical Validation Suite

Report Date: March 24, 2026
BBRES-RNG Developer: Mehul Singh
Significance Level: $\alpha = 0.01$ (99% Confidence)

Table of Contents

1. Executive Summary	
2. Test Methodology	
2.1 Test Parameters	
2.2 Test Categories	
3. NIST SP 800-22 Core Tests	
4. Extended NIST and Uniformity Tests	
4.1 Extended NIST Tests	
4.2 Distribution and Uniformity Analysis	
5. Spectral, Entropy, and Structural Analysis	
5.1 Spectral Analysis (DFT Periodicity Test)	
5.2 Entropy Analysis	
5.3 Autocorrelation Profile	
6. Adversarial and Cryptographic Tests	
6.1 Adversarial Predictability Tests	
6.2 SHA-256 CSPRNG Wrapper Validation	
7. Integer Distribution and Sequence Tests	
7.1 Integer Distribution Tests	
7.2 Integer Sequence Tests	
8. Cross-Segment Consistency	
9. Cryptographic Suitability Assessment	
10. Failed Test Analysis: Java Math.random()	
10.1 Failure: Maurer's Universal Statistical Test	
10.2 Failure: Gap Test (KS)	
11. Conclusions	
11.1 Summary of Findings	
11.2 Usage Guidance	
11.3 Closing Statement	

1. Executive Summary

This report presents a comparative statistical analysis of three random number generator (RNG) architectures subjected to an identical battery of 30 tests. The generators evaluated are: BBRES-RNG (Bit Based Randomised Entropy System), a scheduler-based entropy harvesting RNG developed by Mehul Singh; Java SecureRandom, the standard cryptographically secure pseudo-random number generator (CSPRNG) in the Java platform; and Java Math.random(), a linear congruential generator (LCG) provided in the Java standard library.

Each generator was tested against the same parameters: 2,000,000-bit samples for bitstream analysis, 100,000-integer samples for distribution analysis, and a uniform significance threshold of $\alpha = 0.01$ across all tests. The test battery encompassed NIST SP 800-22 core tests, extended statistical tests, uniformity analysis, spectral and entropy evaluation, adversarial predictability attacks, cryptographic wrapper validation, and integer-level distribution and sequence analysis.

The aggregate results are summarised in the table below.

RNG Architecture	Tests Passed	Verdict	Rank	Rating	Crypto-Suitable
BBRES-RNG	30/30	Accepted	#1	★★★★★	Yes
Java SecureRandom	30/30	Accepted	#2	★★★★★	Yes
Java Math.random()	28/30	Review Needed	#3	★★★★☆	No

Both BBRES-RNG and Java SecureRandom achieved a perfect 30/30 pass rate. BBRES-RNG is ranked first on the basis of a marginally higher Maurer's Universal Statistical test p-value (0.9817 versus 0.6299), which reflects lower compressibility in the output bitstream. Java Math.random() failed two tests—Maurer's Universal Statistical test and the Gap Test (KS)—both attributable to the structural limitations inherent in its LCG architecture.

2. Test Methodology

All three RNG architectures were tested under identical conditions. The test parameters and categorisation are detailed below.

2.1 Test Parameters

Parameter	Value
Bit sample size	2,000,000 bits per RNG
Integer sample size	100,000 integers per RNG
Integer range	[0..999] for BBRES and Math.random(); [0..998] for SecureRandom
Significance level	$\alpha = 0.01$ (99% confidence)
Total tests per RNG	30 (90 total across all three)
Analysis date	March 24, 2026

2.2 Test Categories

The 30 tests are organised into eight categories: NIST SP 800-22 Core (9 tests), Extended NIST (3 tests), Uniformity (3 tests), Spectral (1 test), Adversarial (3 tests), Cryptographic (1 test), Integer Distribution (6 tests), and Integer Sequence (4 tests). Each test yields a p-value; values below $\alpha = 0.01$ indicate a statistically significant departure from ideal random behaviour.

3. NIST SP 800-22 Core Tests

The NIST Special Publication 800-22 defines a widely adopted battery of statistical tests for evaluating randomness quality. All three RNGs passed all nine core NIST tests, with all p-values exceeding the 0.01 threshold.

Test	BBRES	SecureRandom	Math.random()	Result
Frequency (Monobit)	0.0943	0.4033	0.5813	PASS
Block Frequency (M=128)	0.9238	0.8189	0.5744	PASS
Runs Test	0.3233	0.1434	0.4217	PASS
Longest Run of Ones	0.4692	0.0205	0.4076	PASS
Cumulative Sums (Fwd)	0.1780	0.5785	0.2012	PASS
Cumulative Sums (Rev)	0.1637	0.3729	0.5490	PASS
Approx. Entropy (m=2)	0.0537	0.4942	0.8708	PASS
Serial (m=2) — δ_1	0.1520	0.2411	0.6221	PASS
Serial (m=2) — δ_2	0.3250	0.1429	0.4218	PASS

No anomalies were observed in the core NIST suite for any of the three generators.

4. Extended NIST and Uniformity Tests

4.1 Extended NIST Tests

Extended tests assess compressibility (Maurer's Universal), block uniformity (Poker test), and cycle behaviour (Random Excursion Variant). Java Math.random() recorded a failure on Maurer's Universal Statistical test with a p-value of 0.0083, falling below the 0.01 threshold.

Test	BBRES	SecureRandom	Math.random()	Result
Maurer's Universal	0.9817	0.6299	0.0083	BBRES/SR: PASS; MR: FAIL
Poker Test (m=4)	0.4792	0.4373	0.2372	ALL PASS
Random Excursion Var.	0.5000	0.9622	0.5000	ALL PASS

4.2 Distribution and Uniformity Analysis

Uniformity tests verify that all possible bit patterns occur with expected frequency at byte, nibble, and 2-bit granularities. All three generators passed all uniformity tests.

Test	BBRES	SecureRandom	Math.random()
Byte-Level Chi-Square	0.4681	0.7584	0.3183
Nibble-Level Chi-Square	0.4792	0.4373	0.2372
2-Bit Pair Distribution	0.3869	0.4210	0.9353

5. Spectral, Entropy, and Structural Analysis

5.1 Spectral Analysis (DFT Periodicity Test)

The Discrete Fourier Transform test evaluates the bitstream for periodic components. All three generators produced output with no detectable periodicity, with all p-values comfortably above the threshold.

Metric	BBRES	SecureRandom	Math.random()
FFT p-value	0.2275	0.1813	0.3240
Peaks below threshold	950,186 / 1M	950,206 / 1M	950,152 / 1M
Result	PASS	PASS	PASS

5.2 Entropy Analysis

Entropy measures the information density and unpredictability of the output. All three generators achieved Shannon entropy above 99.99% of the theoretical 8-bit maximum, indicating near-ideal output characteristics.

Metric	BBRES	SecureRandom	Math.random()	Theoretical Max
Shannon Entropy (8-bit)	7.99926	7.99931	7.99923	8.00000
% of Maximum	99.991%	99.991%	99.990%	100%
Min-Entropy (8-bit)	7.8831	7.8283	7.8790	8.00000
Transition Rate	0.5003	0.4995	0.5003	0.5000

5.3 Autocorrelation Profile

Autocorrelation measures the linear relationship between bits at various lag distances. Values near zero indicate no serial dependency. All three generators exhibited negligible autocorrelation ($|r| < 0.002$ at all tested lags from 1 to 256).

6. Adversarial and Cryptographic Tests

6.1 Adversarial Predictability Tests

These tests attempt to predict subsequent output using frequency-based pattern matching and machine learning (logistic regression with window sizes of 8 and 16 bits). Prediction accuracy near 50% indicates unpredictable output. All three generators passed all adversarial tests.

Test	BBRES	SecureRandom	Math.random()
Freq. Prediction (w=8)	p=0.8842, Acc=50.45%	p=0.5026, Acc=50.48%	p=0.8210, Acc=50.46%
ML Attack (LogReg, w=16)	p=0.5793, Acc=49.90%	p=0.0548, Acc=50.80%	p=0.5714, Acc=49.91%
Pattern Repetition (w=53)	p=1.0, None	p=1.0, None	p=1.0, None

6.2 SHA-256 CSPRNG Wrapper Validation

Each generator's output was seeded into a SHA-256-based CSPRNG wrapper, and the resulting output was re-tested for predictability. All three generators passed this test. It is noted, however, that passage of the wrapper test does not confer cryptographic suitability on the underlying source; the SHA-256 transformation compensates for input weaknesses.

RNG	p-value	Post-Hash Acc.	Result
BBRES-RNG	1.0000	53.00%	PASS
Java SecureRandom	1.0000	52.97%	PASS
Java Math.random()	1.0000	53.40%	PASS

7. Integer Distribution and Sequence Tests

7.1 Integer Distribution Tests

Integer output was tested for distributional uniformity using Chi-Square, mean Z-test, variance analysis, binned goodness-of-fit, birthday spacing, and coupon collector tests. All three generators passed all six tests.

Test	BBRES	SecureRandom	Math.random()
Chi-Square Uniformity	p=0.4752	p=0.2856	p=0.4587
Mean (Z-test)	p=0.3989	p=0.6638	p=0.0718
Variance (Chi-sq)	p=0.8426	p=0.8731	p=0.8595
Binned Goodness-of-Fit	p=0.7563	p=0.0753	p=0.5551
Birthday Spacing	p=1.0000	p=1.0000	p=1.0000
Coupon Collector	p=0.5000	p=0.5000	p=0.5000

7.2 Integer Sequence Tests

Sequence tests verify that consecutive integer outputs show no patterns, memory effects, or predictability. Java Math.random() failed the Gap Test (KS) with a p-value of 0.0005, indicating statistically significant deviation in the spacing between recurrences of specific integer values.

Test	BBRES	SecureRandom	Math.random()
Runs Up/Down	p=0.9940 PASS	p=0.7564 PASS	p=0.5651 PASS
Lag-1 Autocorrelation	p=0.4181 PASS	p=0.1351 PASS	p=0.1571 PASS
Gap Test (KS)	p=0.3326 PASS	p=0.9813 PASS	p=0.0005 FAIL
Permutation (t=5)	p=0.7034 PASS	p=0.0912 PASS	p=0.6960 PASS

8. Cross-Segment Consistency

Each bitstream was divided into 10 equal segments of 200,000 bits. Each segment was independently tested for bit balance. A Kolmogorov-Smirnov test on segment p-values was applied to verify uniformity of quality across the entire stream, and cross-half correlation was computed to assess independence between the first and second halves.

Metric	BBRES	SecureRandom	Math.random()
KS Statistic	0.2515	0.1431	0.1871
KS p-value	0.4766	0.9689	0.8139
Cross-half correlation	-0.000675	-0.001378	-0.000722
Assessment	Consistent	Consistent	Consistent

All three generators demonstrated consistent quality across all segments, with no evidence of degradation or dependency between stream halves.

9. Cryptographic Suitability Assessment

Cryptographic suitability is assessed by evaluating the overall pass rate, entropy quality, resistance to predictability attacks, compressibility characteristics, and the underlying algorithmic architecture of each generator.

Criteria	BBRES-RNG	SecureRandom	Math.random()
Overall pass rate	30/30 (100%)	30/30 (100%)	28/30 (93.3%)
Maurer's Universal	p=0.9817 PASS	p=0.6299 PASS	p=0.0083 FAIL
Shannon Entropy (% max)	99.991%	99.991%	99.990%
Min-Entropy (% max)	98.54%	97.85%	98.49%
ML Attack resistance	49.90% acc	50.80% acc	49.91% acc
SHA-256 wrapper	PASS	PASS	PASS
Architecture	Scheduler entropy	OS entropy + DRBG	Linear Congruential
Crypto verdict	Suitable	Safe (Standard)	Not Suitable

BBRES-RNG and Java SecureRandom both satisfy the criteria for cryptographic suitability. Java Math.random(), based on an LCG architecture, does not meet the requirements for cryptographic use.

10. Failed Test Analysis: Java Math.random()

Java Math.random() failed two of the 30 tests. Both failures are attributable to the underlying Linear Congruential Generator algorithm, which computes successive values via the recurrence $X(n+1) = (a \times X(n) + c) \bmod m$.

10.1 Failure: Maurer's Universal Statistical Test

Attribute	Detail
p-value	0.008287 (below $\alpha = 0.01$)
fn statistic	6.1911 (expected closer to 6.1953)
Interpretation	The bitstream exhibits detectable compressibility patterns.
Root cause	LCG produces output with structural regularity in long-range bit patterns due to the linear recurrence relation.

10.2 Failure: Gap Test (KS)

Attribute	Detail
p-value	0.000540 (well below $\alpha = 0.01$)
Mean gap	1792.78 (expected: 1000)
Interpretation	Spacing between recurrences of specific integer values deviates significantly from the expected geometric distribution.
Root cause	LCG modular arithmetic creates subtle clustering and spacing patterns in the integer output domain.

11. Conclusions

11.1 Summary of Findings

BBRES-RNG achieved a perfect 30/30 pass rate across all test categories. Its Maurer's Universal p-value of 0.9817 was the highest among the three generators, suggesting low compressibility in the output bitstream. The scheduler-based entropy harvesting architecture produced output that was statistically indistinguishable from ideal random across all evaluated metrics.

Java SecureRandom also achieved a 30/30 pass rate, consistent with its established role as a purpose-built CSPRNG in the Java ecosystem. It recorded the highest Shannon entropy (7.99931) and the strongest Gap Test result ($p = 0.9813$). It remains a well-established, FIPS-aligned solution for cryptographic applications.

Java Math.random() scored 28/30, failing Maurer's Universal Statistical test and the Gap Test (KS). These failures are inherent to the LCG architecture and cannot be resolved without modifying the underlying algorithm. The generator is not suitable for cryptographic or high-assurance applications but remains adequate for general-purpose use cases where cryptographic strength is not required.

11.2 Usage Guidance

Use Case	Recommended	Acceptable?
Cryptographic key generation	BBRES-RNG or SecureRandom	Not Math.random()
Session tokens / nonces	BBRES-RNG or SecureRandom	Not Math.random()
Lottery / fairness-critical	BBRES-RNG or SecureRandom	Not Math.random()
Monte Carlo simulations	Any of the three	All acceptable
Game mechanics / UI	Any of the three	All acceptable
Statistical sampling	Any of the three	All acceptable

11.3 Closing Statement

Based on the 30-test statistical validation suite applied under uniform conditions, BBRES-RNG and Java SecureRandom both demonstrate output quality consistent with cryptographic-grade entropy sources. BBRES-RNG is ranked first due to marginal advantages in compressibility resistance as measured by the Maurer's Universal test. Java SecureRandom remains a proven and widely adopted standard. Java Math.random() is appropriate for non-security contexts but should not be used where cryptographic assurance is required.

Disclaimer: This report is an independent analysis based on the submitted validation data for each RNG architecture. All tests used a significance level of $\alpha = 0.01$ (99% confidence). Sample sizes: 2,000,000 bits and 100,000 integers per RNG. Report generated: March 24, 2026. BBRES-RNG developer: Mehul Singh.